

CS 340 Project Two: Grazioso Salvare Dashboard

Sydney Brown

CS 340: Client Server Development

Southern New Hampshire University

Professor Helen

August 16, 2026

Functionality Description

This dashboard was built for Grazioso Salvare, a rescue animal company that needs a way to sort through dogs in the Austin Animal Center (AAC) dataset and find ones that fit specific rescue training profiles. The whole point is to make that filtering process fast and visual instead of manually digging through data.

The dashboard has four filter options that the user selects from a set of radio buttons:

- Water Rescue - pulls Labrador Retriever Mix, Chesapeake Bay Retriever, and Newfoundland dogs; Intact Female; between 26 and 156 weeks old.
- Mountain or Wilderness Rescue - pulls German Shepherd, Alaskan Malamute, Old English Sheepdog, Siberian Husky, and Rottweiler dogs; Intact Male; between 26 and 156 weeks old.
- Disaster or Individual Tracking - pulls Doberman Pinscher, German Shepherd, Golden Retriever, Bloodhound, and Rottweiler dogs; Intact Male; between 20 and 300 weeks old.
- Reset - clears everything and brings back the full dataset.

When a filter is selected, the data table updates automatically to show only the matching animals.

Clicking a row on the table updates the geolocation map to drop a marker at that animal's location, with the breed showing on hover and the animal's name in a popup. There's also a pie chart that updates with the table and shows the breed breakdown of whatever filter is currently active.

Tools Used and Why

MongoDB was used as the database. It stores the animal shelter records and handles all the querying. In the MVC setup for this project, MongoDB is the Model, and it's where the data

lives and gets pulled from. A custom Python CRUD module (AnimalShelter class in `CRUD_Python_Module.py`) sits between the database and the dashboard and handles all the read operations.

Dash was used to build the actual dashboard interface. It handles both the View side (the layout, the table, radio buttons, chart, and map) and the Controller side (the callback functions that listen for user input and update the components). Dash was a good fit here because it lets you build a fully interactive web app in Python without needing to write separate HTML, CSS, or JavaScript. The geolocation map specifically used the dash-leaflet library.

Steps Taken

- Fixed the import statement to match the actual CRUD module filename and class name (`CRUD_Python_Module`, `AnimalShelter`).
- Updated the database connection in `CRUD_Python_Module.py` to connect to the local MongoDB instance correctly given the Codio environment configuration.
- Added the Grazioso Salvare logo and my name (Sydney Brown) to the dashboard layout.
- Set up the `DataTable` with `page_size`, `sort_action`, `row_selectable`, and `selected_rows`.
- Built out the radio button filter widget with all four options and Reset as the default.
- Wrote the `update_dashboard` callback to hit MongoDB with the right query depending on which filter is selected, using the breed lists, sex, and age ranges from the Dashboard Specifications Document.
- Added the `update_graphs` callback to generate a pie chart showing breed distribution for the current filtered dataset.

- Brought in the geolocation map callback from the Module Six Milestone and connected it to the table's row selection.
- Added a None check to the update_styles callback so it doesn't crash when nothing is selected.
- Tested all four filters and Reset, confirmed the table, chart, and map all update correctly, and took the required screenshots.

Challenges Faced

The main challenges came down to debugging issues that came up during the build. When callback functions were copied over from the Module Six Milestone into the Project Two file, the indentation didn't transfer cleanly, so several if/elif/else blocks and return statements ended up at the wrong level and had to be corrected manually.

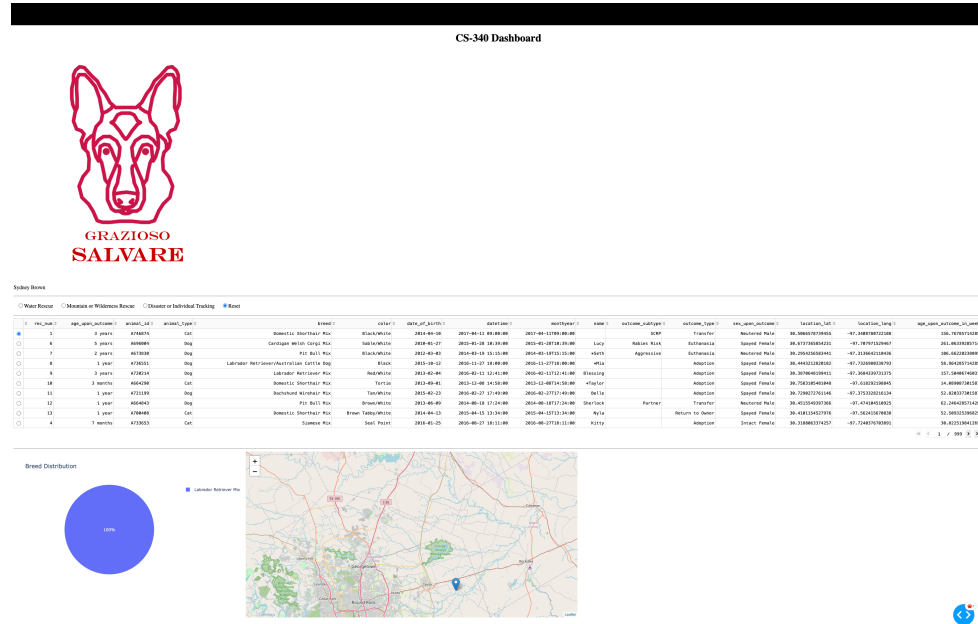
There were also a few variable name mismatches that had to be caught and fixed. For example, the callback was referencing shelter instead of db, which is what the database connection was actually named. Another one was the update_dashboard callback only having one Output declared in the decorator when it was actually returning two values (data and columns), which caused the table to break until that was corrected.

The Codio environment also had a MongoDB configuration issue where authentication wasn't being enforced server-side, which required updating the connection string to match what the environment was actually doing.

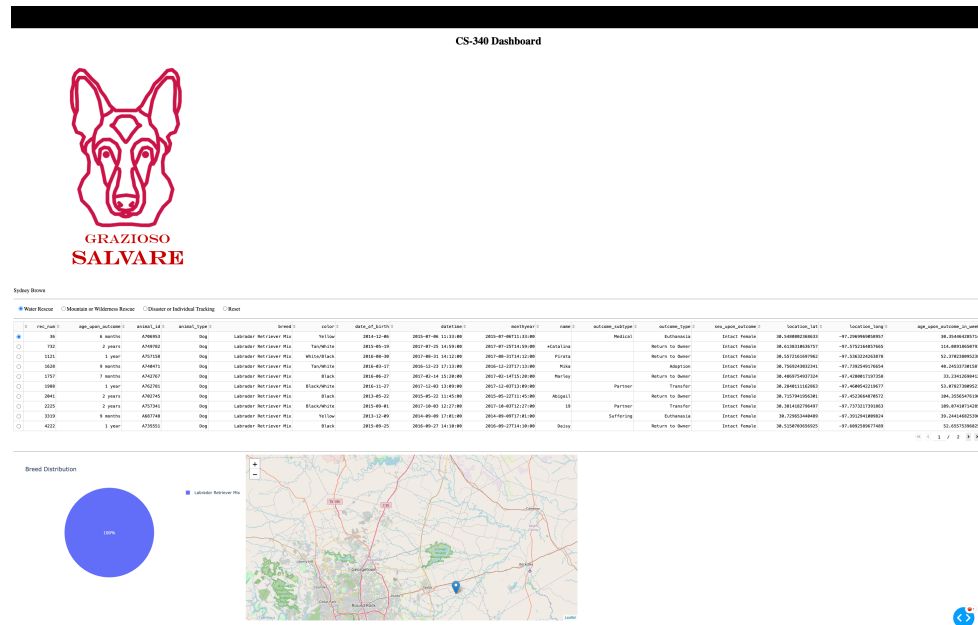
Screenshots

Screenshots below show the dashboard in each of its five states.

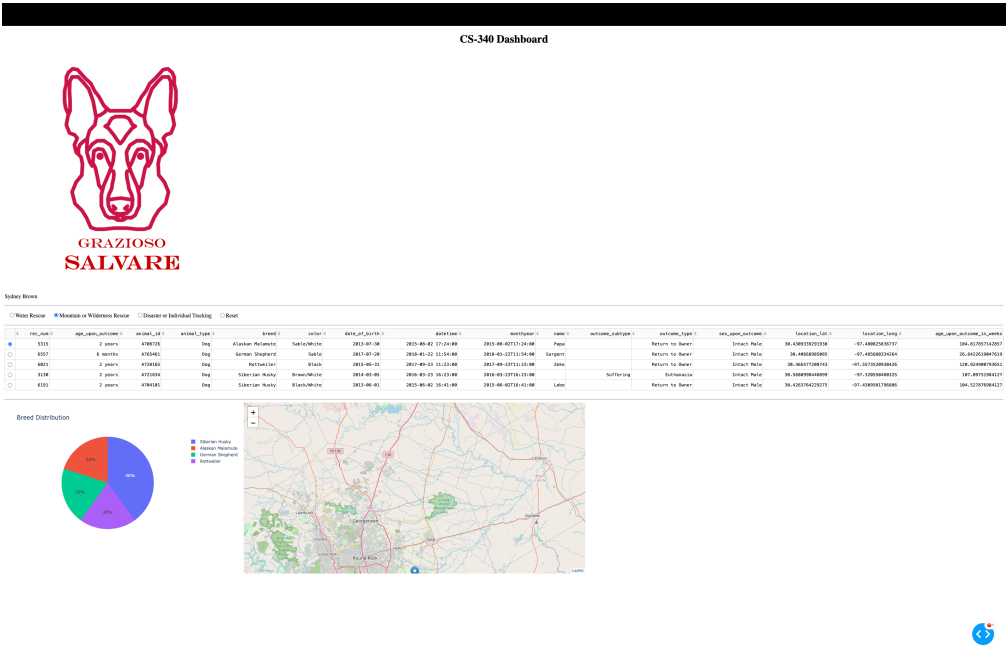
Screenshot 1: Starting State (Reset — Full Dataset)



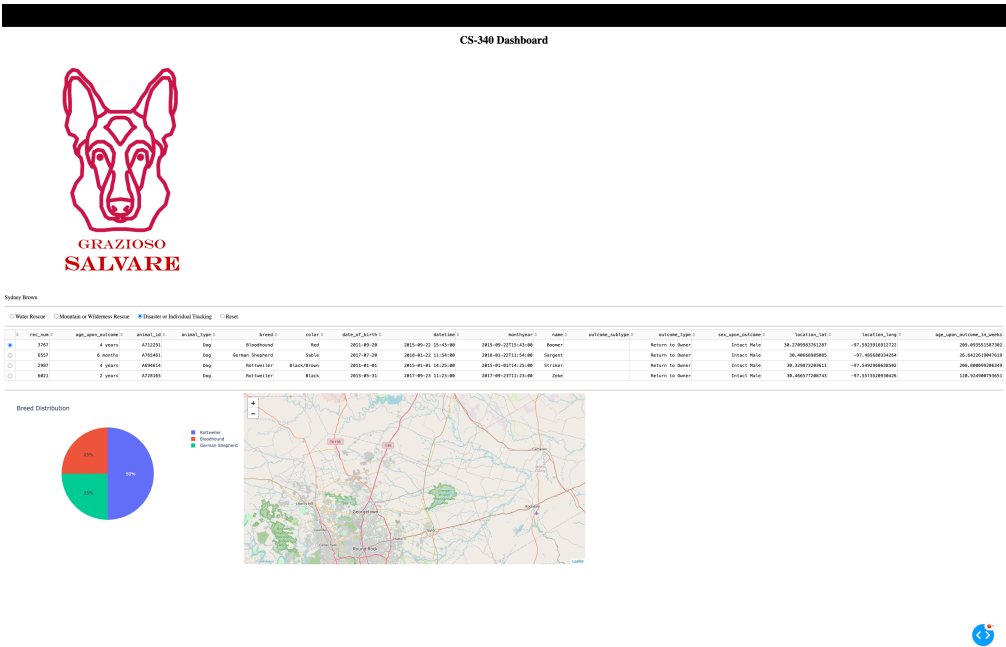
Screenshot 2: Water Rescue Filter



Screenshot 3: Mountain or Wilderness Rescue Filter



Screenshot 4: Disaster or Individual Tracking Filter



Screenshot 5: Reset Filter

CS-340 Dashboard



Sydney Brown

[illegible]